

nxt-backend API reference

Base URL (local dev): `http://localhost:3000` (Android emulator: replace `localhost` with `10.0.2.2`, per `nxt-app/constants/apiUrl.ts`).

Authentication

Auth is handled by [Better Auth](#), mounted at `/api/auth/*` (`src/auth/auth.ts`, `src/main.ts`). Every route below is **protected by default** — a global `AuthGuard` (`src/auth/auth.guard.ts`) rejects unauthenticated requests with `401 Unauthorized`, unless the route is explicitly marked `@Public()`.

Sessions are cookie-based (`better-auth.session_token`). The mobile app should use `@better-auth/expo`'s client, which stores the session in `SecureStore` and attaches it to requests automatically — no manual token handling needed.

Auth endpoints (`/api/auth/*`)

These are provided entirely by Better Auth — no NestJS controller code. Full reference: [Better Auth API docs](#).

Method	Path	Description
POST	<code>/api/auth/sign-up/email</code>	Register with email + password. Body: <code>{ email, password, name }</code> .
POST	<code>/api/auth/sign-in/email</code>	Log in with email + password. Body: <code>{ email, password }</code> .
POST	<code>/api/auth/sign-in/social</code>	Start an OAuth flow. Body: <code>{ provider: "google", callbackURL }</code> . Returns <code>{ url, redirect: true }</code> — open url in a browser.
GET	<code>/api/auth/callback/google</code>	Google OAuth callback (browser redirect target, not called directly by app code).
GET	<code>/api/auth/get-session</code>	Returns the current <code>{ session, user }</code> for the request's cookie, or <code>null</code> .
POST	<code>/api/auth/sign-out</code>	Ends the current session.

`provider: "apple"` is not wired up yet — no Apple Developer Program membership on the account yet.

Identity vs. profile

`sign-up/sign-in` create/return an **AuthUser** (`id`, `email`, `name`, ...) — this is just the login identity. It has no marketplace data (no `placeId`, `rating`, etc.) until the client calls `POST /users` (see below) to create the matching profile row. `User.id` is always equal to the `AuthUser.id` of the account that owns it (shared primary key), so a session's `user.id` can be used directly as a profile id — no separate lookup needed.

A freshly signed-up account has **no** `User` profile row yet — routes that read `req.user` as a profile (none currently) would need to handle that; currently only `POST /users` (onboarding) assumes the profile doesn't exist yet.

External services & where to find credentials

Quick reference for where each third-party service is managed and where its keys live in this repo, so you don't have to hunt for them later.

Neon (Postgres database)

- **Console:** console.neon.tech — project region `eu-central-1` (visible in the connection host, `...c-4.eu-central-1.aws.neon.tech`). Log in with whichever account/provider (GitHub/Google/email) the project was created under; the project list is scoped to that account, there's no separate project-id URL to bookmark.
- **Used via:** `@prisma/adapters-neon` (Neon's serverless driver), wired up in `src/prisma/prisma.service.ts` and `src/auth/auth.ts`.
- **Credential:** `DATABASE_URL` in `nxt-backend/.env`. If it needs rotating, get a fresh connection string from **Dashboard** → **Connect** (or **Settings** → **Connection Details**) — pick the **pooled** connection (hostname has `-pooler` in it, like the one currently in `.env`), not the direct one.
- **Good to know:**
 - Free-tier compute **auto-suspends after a few minutes of inactivity**. The first query after a period of no traffic will be noticeably slower (a "cold start" as it wakes back up) — that's expected, not a bug.
 - Neon supports **branching** — you can create a full copy-on-write branch of the database (schema + data) straight from the console, point a local `.env` at it, and test risky migrations there without touching the real data. Worth using before any migration you're unsure about.
 - `.env` has a commented-out `DATABASE_URL_UNPOOLED` line — only needed for things like advisory locks or long transactions that don't play well through the pooler. Not currently used anywhere in this codebase.
 - If the app suddenly can't connect, check the Neon console for a usage/quota banner before assuming it's a code problem — free-tier projects have storage/compute limits.

Cloudflare R2 (image storage)

- **Console:** dash.cloudflare.com/?to=/:account/r2/overview — Cloudflare's standard "jump straight to R2" link, works for whichever account you're logged into. Bucket: `nxt-door-images`.
- **Used via:** `@aws-sdk/client-s3` (R2 is S3-compatible), all in `src/uploads/uploads.service.ts`.
- **Credentials** — all in `nxt-backend/.env`:

Env var	Where to find it in the Cloudflare dashboard
<code>R2_ACCOUNT_ID</code>	Right sidebar of any R2 page, or Account Home
<code>R2_ACCESS_KEY_ID</code> / <code>R2_SECRET_ACCESS_KEY</code>	R2 → Manage API tokens → create/view token (secret is only shown once at creation — if lost, issue a new token)
<code>R2_BUCKET_NAME</code>	The bucket name as created (<code>nxt-door-images</code>)

R2_ENDPOINT	R2 bucket → Settings — format <code>https://<account_id>.r2.cloudflarestorage.com</code>
R2_PUBLIC_URL	R2 bucket → Settings → Public access (the <code>r2.dev</code> dev URL, or a custom domain if one's attached)

- **Good to know:**

- R2's selling point over S3 is **zero egress fees** — serving images out to the app doesn't rack up bandwidth charges the way S3 would.
- The API token should be scoped to **just this bucket** with **Object Read & Write** permission (not account-wide) — check this under **Manage API tokens** if you ever need to reissue one, for least-privilege's sake.
- Public access is **off by default per bucket** and has to be explicitly turned on (Settings → Public Access) — that's what generates the `r2.dev` URL used as `R2_PUBLIC_URL`. A custom domain can be attached there later instead.
- All uploads currently go *through this backend* (`uploads.service.ts` holds the credentials server-side) — the app never talks to R2 directly, so there's no CORS configuration to worry about. That would only become relevant if a future direct-browser-to-R2 (presigned URL) upload flow gets added.

Google Auth (OAuth sign-in)

- **Console:** console.cloud.google.com → **Google Auth Platform** (Google's newer name for what used to be "OAuth consent screen"), left sidebar:
- **Branding** — app name, logo, support email shown on the consent screen.
- **Audience** — publishing status (Testing/Production) and the test-user allowlist (required while still in Testing).
- **Clients** — the actual OAuth Client ID/Secret. Click the existing Web client to view or rotate the secret.
- **Used via:** `socialProviders.google` in `src/auth/auth.ts`.
- **Credentials:** `GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` in `nxt-backend/.env`. Authorized redirect URI on the Google side must match `BETTER_AUTH_URL + /api/auth/callback/google`.
- Apple Sign-In isn't set up yet (needs a paid Apple Developer Program membership) — see developer.apple.com when that's ready.

Google Maps

- **Console:** same console.cloud.google.com project as Google Auth above → **APIs & Services** → **Credentials** → **API keys** (a plain API key, not an OAuth client — different section of the same page).
- **Used via:** `react-native-maps` in `nxt-app` (not this backend).
- **Key location:** `nxt-app/app.json` → `expo.android.config.googleMaps.apiKey`. **Android only, on purpose** — `react-native-maps` isn't given an explicit provider anywhere in the app, so on iOS it defaults to Apple's own MapKit (no API key needed at all); only Android has no built-in maps provider and requires the Google Maps SDK key. Unlike the OAuth secret, this key ends up bundled into the app itself (that's normal for Maps keys), so it should be locked down in the Cloud Console via **Application restrictions** (Android package name + SHA-1 fingerprint) and **API restrictions** (Maps SDK only) rather than treated as a true secret.

Finding all Google Cloud keys in one place

Everything above (the OAuth client and the Maps API key) lives in the same Google Cloud project and shows up together on one page: console.cloud.google.com → **APIs & Services** → **Credentials**. That page lists every credential regardless of type — an **"API keys"** section (Maps and any other plain keys) and an **"OAuth 2.0 Client IDs"** section (the Google Auth client) side by side. If a key ever goes missing from `.env/app.json`, this is the one page to check first rather than hunting through Branding/Audience/Clients individually.

Conventions

- **■ Public** — no session required.
- **■ Auth required** — 401 if no valid session.
- **■■ Auth + ownership** — 401 if no session, 403 if the session's user doesn't own the resource being modified.
- All POST/PATCH bodies are validated with `class-validator` (`whitelist: true` — unknown fields are silently stripped, not rejected).
- Errors follow Nest's default shape: `{ "statusCode": number, "message": string | string[], "error": string }`.

Listings

Resource: `src/listings/`. Model: `Listing` (Prisma).

	Method	Path	Auth
Create	POST	<code>/listings</code>	■
List	GET	<code>/listings</code>	■
Get one	GET	<code>/listings/:id</code>	■
Update	PATCH	<code>/listings/:id</code>	■■
Delete	DELETE	<code>/listings/:id</code>	■■

POST `/listings`

Creates a listing owned by the current session user (`userId` is taken from the session — not accepted in the body).

Body (`CreateListingDto`):

```
{
  "title": "string",
  "description": "string",
  "price": 10,
  "currency": "USD",
  "images": ["https://..."],
  "side": "offer", // "offer" | "order"
  "category": "string",
  "status": "active", // optional, "active" | "closed" | "reserved" | "disabled",
```

```
default "active"
"latitude": 50.45,
"longitude": 30.52,
"address": "string"
}
```

Response 201: the created Listing row (includes generated id, userId, createdAt, updatedAt).

GET /listings

Query params (FindListingsQueryDto), all optional:

Param	Type	Notes
side	"offer" "order"	
userId	string	filter by owner — a read-only filter, not an identity check, so any value is allowed
limit	int, 1–100	default 20
offset	int, ≥0	default 0

Response 200: Listing[], newest first.

GET /listings/:id

Response 200: Listing. 404 if not found.

PATCH /listings/:id

Body: any subset of CreateListingDto fields. 403 if the session user isn't the listing's owner. If images is included and some URLs were removed, the removed files are deleted from R2 automatically.

DELETE /listings/:id

403 if not the owner. Deletes the listing's images from R2 before deleting the row.

Saved listings

Resource: src/saved-listings/. Model: SavedListing. Always scoped to **the current session user** — there is no way to read or modify another user's saved listings through this API.

	Method	Path	Auth
Save	POST	/saved-listings	■
List mine	GET	/saved-listings	■

Get one	GET	/saved-listings/:id	■
Unsave	DELETE	/saved-listings/:id	■■

POST /saved-listings

Body: { "listingId": 1 } (userId comes from the session). 409 Conflict if this user already saved this listing (unique constraint on [userId, listingId]).

GET /saved-listings

Response 200: the current user's saved listings, each with the nested listing included.

GET /saved-listings/:id

Response 200: a SavedListing row by its own id. 404 if not found. (Not currently ownership-checked — anyone authenticated can look up a saved-listing row by id if they know it.)

DELETE /saved-listings/:id

404 if the row doesn't exist, 403 if it belongs to a different user.

Users (profiles)

Resource: src/users/. Model: User — the marketplace profile, id shared with AuthUser.id.

	Method	Path	Auth
Onboard (create profile)	POST	/users	■
Get profile	GET	/users/:id	■
Update profile	PATCH	/users/:id	■■
Delete profile	DELETE	/users/:id	■■

POST /users

Called once, right after sign-up, to finish onboarding. id is taken from the session (equal to the AuthUser.id) — not accepted in the body.

Body (CreateUserDto):

```
{
  "name": "string",
  "avatar": "https://...",
  "placeId": "string" // must reference an existing Place
}
```

Response 201: the created `User` row (`onboarded: false`, `rating: 0`, etc. by default).

GET `/users/:id`

Public profile lookup. Response 200: `User`. 404 if the `id` has no profile (e.g. signed up but hasn't onboarded yet).

PATCH `/users/:id`

Body: any subset of `CreateUserDto` fields. 403 if `:id` isn't the session user's own id.

DELETE `/users/:id`

403 if `:id` isn't the session user's own id.

Places

Resource: `src/places/`. Model: `Place` (residential buildings/complexes users belong to). Reads are public (needed to show the place picker during onboarding, before the user necessarily has a session — in practice they do, but there's no reason to gate it).

	Method	Path	Auth
Create	POST	<code>/places</code>	■
List	GET	<code>/places</code>	■
Get one	GET	<code>/places/:id</code>	■
Update	PATCH	<code>/places/:id</code>	■
Delete	DELETE	<code>/places/:id</code>	■

Note: write routes here have no ownership or role check — any authenticated user can create/edit/delete a `Place` right now. There's no admin/role system yet; if places should be curated rather than user-editable, that needs an authorization rule added before this is exposed to real users.

POST `/places`

Body (`CreatePlaceDto`):

```
{
  "id": "string", // chosen by the caller, not generated
  "name": "string",
  "address": "string",
  "buildings": ["string"],
  "latitude": 50.45,
  "longitude": 30.52,
  "boundary": [{ "latitude": 50.45, "longitude": 30.52 }, ...],
}
```

```
"mgmtPhone": "string?",
"securityPhone": "string?",
"elevatorEmergency": "string?"
}
```

GET /places, **GET** /places/:id

Response: Place / Place[].

PATCH /places/:id, **DELETE** /places/:id

Standard partial update / delete.

Uploads

Resource: src/uploads/. Uploads a single image to Cloudflare R2.

	Method	Path	Auth
Upload image	POST	/uploads	■

multipart/form-data, field name file. Max size 5 MB, image/* mimetypes only (400 otherwise).

Response 200: { "url": "https://<R2_PUBLIC_URL>/<uuid>-<original filename>" }.

There's no matching DELETE /uploads route — deletion happens as a side effect of PATCH/DELETE on listings (removed/orphaned images are cleaned up from R2 there).

Example flow (verified against a running instance)

```
# 1. sign up (also drops a session cookie)
curl -c cookies.txt -X POST http://localhost:3000/api/auth/sign-up/email \
  -H "Content-Type: application/json" \
  -d '{"email": "me@example.com", "password": "password123", "name": "Me"}'

# 2. onboard — create the marketplace profile for that session
curl -b cookies.txt -X POST http://localhost:3000/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Me", "avatar": "https://example.com/a.png", "placeId": "<existing-place-id>"}'

# 3. create a listing — userId is inferred from the cookie, not sent
curl -b cookies.txt -X POST http://localhost:3000/listings \
  -H "Content-Type: application/json" \
  -d '{"title": "Bike", "description": "desc", "price": 10, "currency": "USD", "images": [], "side": "offer", "category": "misc", "latitude": 1, "longitude": 1, "address": "addr"}'
```